

Course: BIT4107 – Mobile Application Development

Student: BIT/2024/31976

KIOKO PAUL

Project: KilimoPro

Logbook Weeks 1–3

WEEK 1: INTRODUCTION TO MOBILE APPLICATIONS DEVELOPMENT

Objective

Understand the mobile application ecosystem (Android, iOS, cross-platform).

Set up a professional development environment.

Create and run the first Flutter app successfully.

Activities Completed

Installed Flutter SDK, Android Studio, VS Code, Git, Java JDK, Node.js, Python 3.x.

Configured Android Emulator (Pixel2_API30 with GPU acceleration).

Fixed NDK issues by manually installing r27d and updating Gradle path.

Ran flutter doctor to confirm environment setup.

Created first Flutter project (flutter create firstapp).

Customized app by changing title, background color, and font size.

Deployed app to emulator and confirmed KilimoPro appears in app drawer.

Screenshots

```

PS C:\FlutterProjects\kilimopro> flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.44.0, on Microsoft Windows [Version 10.0.26200.8524], locale en-KE)
[✓] Windows Version (Windows 11 or higher, 25H2, 2009)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
[✓] Chrome - develop for the web
[X] Visual Studio - develop Windows apps
    X Visual Studio not installed; this is necessary to develop Windows apps.
      Download at https://visualstudio.microsoft.com/downloads/.
      Please install the "Desktop development with C++" workload, including all of its default components
[✓] Connected device (4 available)
[✓] Network resources

! Doctor found issues in 1 category.

```

Figure 1: Flutter doctor output showing no errors.

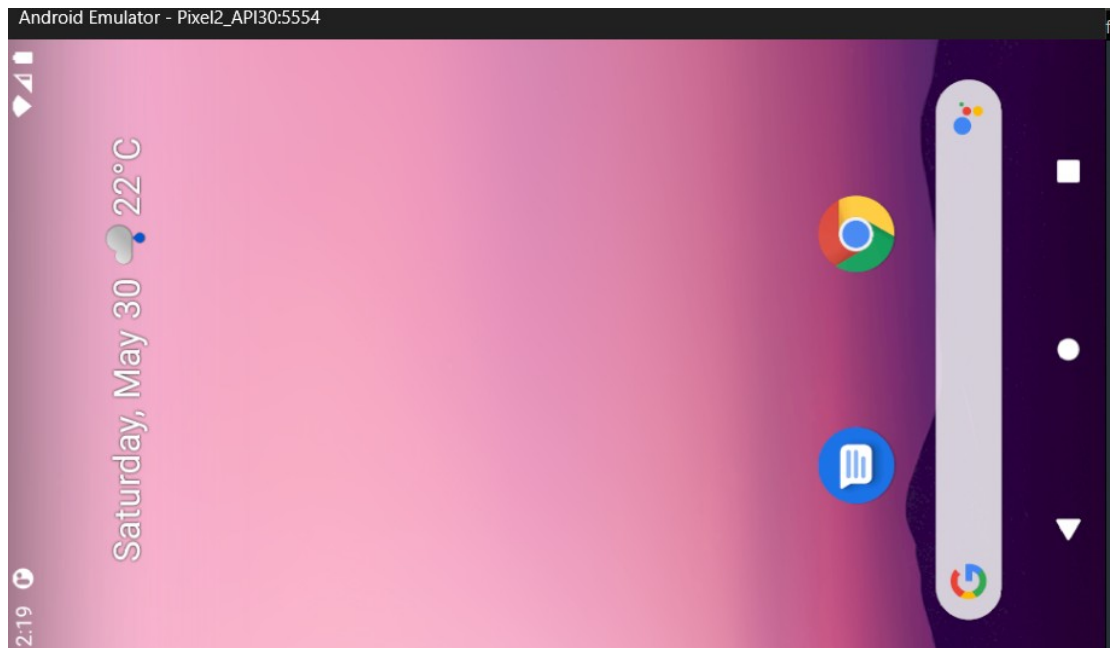


Figure 2: Android Emulator running Pixel2_API30 home screen

```

PS C:\Users\PAUL.DESKTOP-0J907MM> cd $HOME; flutter create hello_world_app
Creating project hello_world_app...
Resolving dependencies in `hello_world_app`... (2.2s)
Downloading packages...
Got dependencies in `hello_world_app`.
Wrote 131 files.

All done!
You can find general documentation for Flutter at: https://flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider: https://www.youtube.com/c/flutterdev

In order to run your application, type:

$ cd hello_world_app
$ flutter run

Your application code is in hello_world_app\lib\main.dart.

```

Figure 3: flutter creating hello world app



Figure 4:Hello world app running in emulator

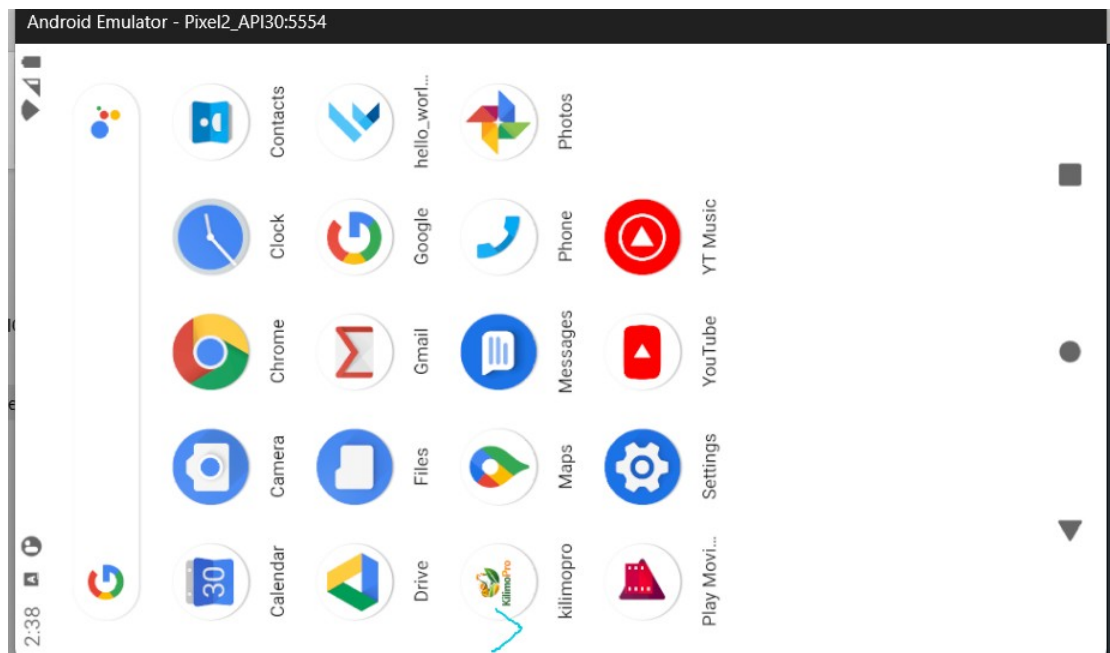


Figure 5::Hello world app title changed to kilimopro; App with background color changed;App with larger font size

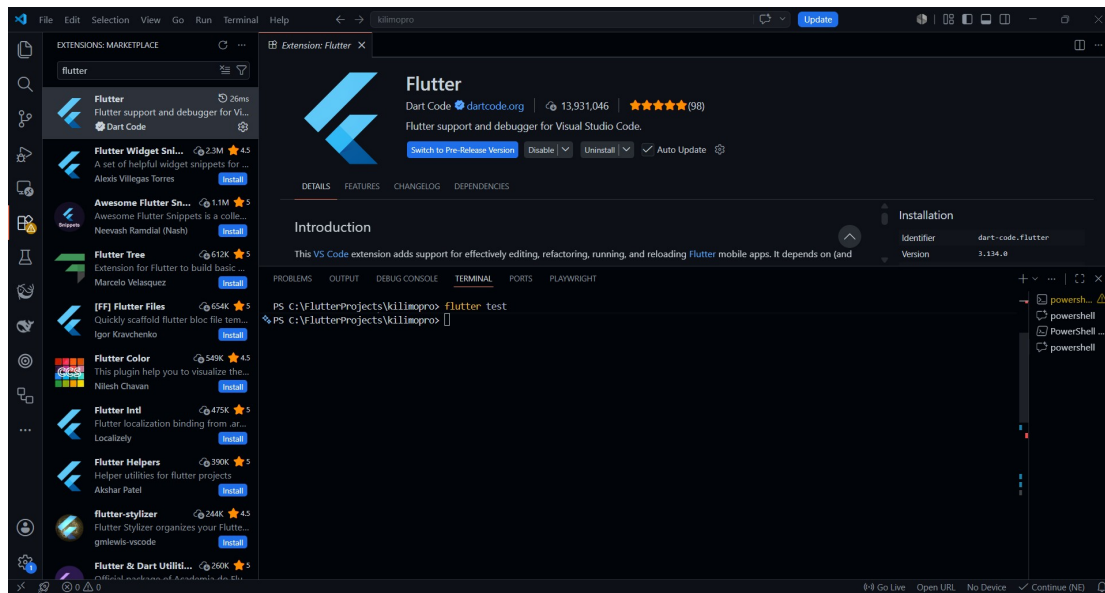


Figure 6:VSCODE with flutter extension installed

Challenges Faced

Emulator freezing due to RAM/heap limits.

Corrupted NDK download (missing source.properties).

VS Code terminal freezing laptop during emulator boot.

Solutions Implemented

Increased emulator RAM/heap in config.ini.

Installed NDK r27d manually and pointed Gradle to correct path.

Started emulator from main CMD instead of VS Code terminal.

WEEK 2: Programming Languages and Frameworks Objective

Practice syntax in Kotlin, Dart, JavaScript, and Python.

Build UI components (buttons, text fields, images, menus, navigation bars).

Create a cross-platform Flutter project.

Implement a Mini Student Management App with login, registration, and local record saving.

Activities Completed

Practiced variables, functions, loops, and classes in Dart.

Created basic UI components in Flutter (buttons, text fields, images).

Built navigation routes between login, registration, and dashboard screens.

Integrated local storage using SQLite (sqflite package) or JSON file.

Tested app flow: login → register student → view dashboard with saved records.

Screenshots

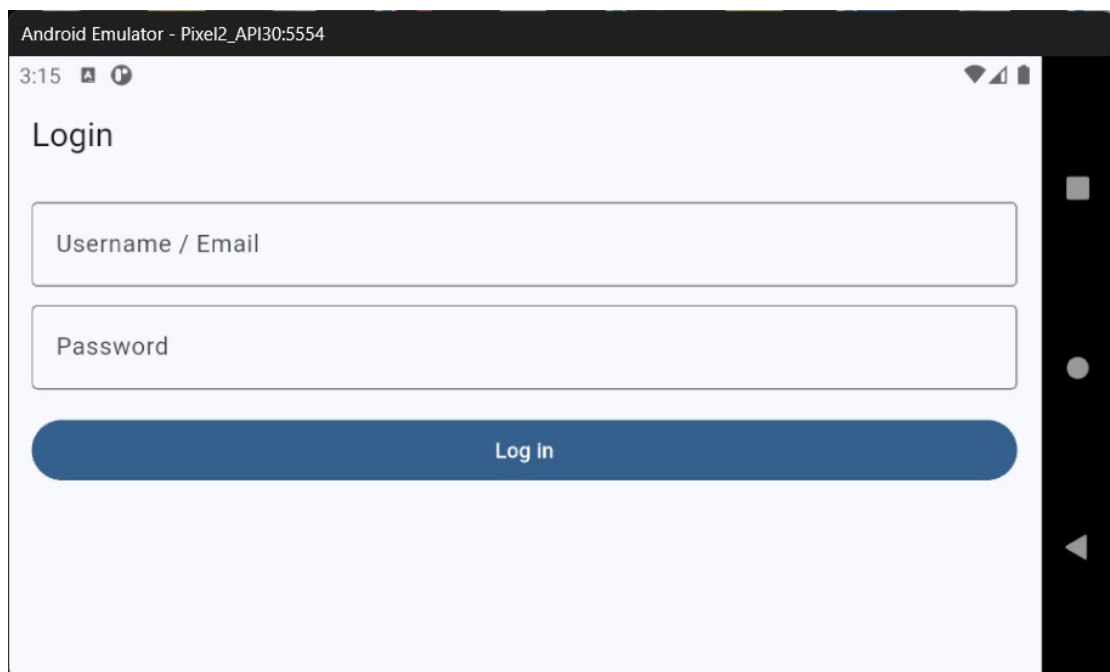


Figure 7: Login screen running in emulator.

Android Emulator - Pixel2_API30:5554

3:19

Register Student

Name

Student ID

Course

Email

Save

Figure 8: Registration fields

Android Emulator - Pixel2_API30:5554

3:22

Dashboard

paul

ID: bit202431976 • Course: bit

+

Figure 9: Dashboard showing saved student records.

```
main.dart X
lib > main.dart > ...
1  import 'package:flutter/material.dart';
2
3  import 'screens/login_screen.dart';
4
5  Run | Debug | Profile
6  void main() {
7    runApp(const MyApp());
8  }
9
10 class MyApp extends StatelessWidget {
11   const MyApp({super.key});
12
13   @override
14   Widget build(BuildContext context) {
15     return MaterialApp(
16       debugShowCheckedModeBanner: false,
17       title: 'Student Management',
18       theme: ThemeData(
19         useMaterial3: true,
20         colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
21       ), // ThemeData
22       home: const LoginScreen(),
23     ); // MaterialApp
24   }
25 }
```

Figure 10: VS Code showing code for navigation/routes.

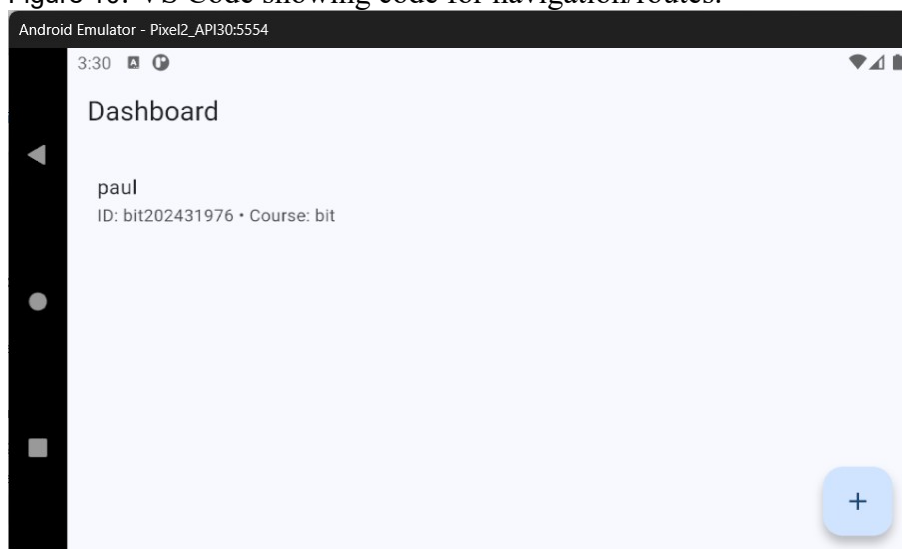


Figure 11: Evidence of local storage (SQLite DB file or JSON data).

Challenges Faced

Setting up navigation between multiple screens.
Handling form validation for login and registration.
Configuring SQLite database in Flutter.

Solutions Implemented

Used Navigator.push and Navigator.pop for screen transitions.
Added input validation using TextFormField with validators.
Installed and configured sqflite package for local database storage.

Outcome

Student Management App runs successfully with login, registration, and dashboard.
Records are saved locally and displayed in the app.
Learned how to connect UI with data storage and navigation.

Reflection

Week 2 introduced real app logic beyond setup.
Practicing multiple languages improved understanding of syntax differences.
Building the student app gave hands-on experience with navigation and local storage.
Ready to move into UI/UX design in Week 3.

Week 3 – User Interface (UI) Development

Objective

Design wireframes for KilimoPro using Figma.

Create GUI designs with color schemes, typography, and responsive layouts.

Convert designs into Flutter code.

Implement UI interactions (validation, alerts, menus, transitions).

Test app on emulator and real device.

Activities Completed

Designed wireframes for 5 KilimoPro screens: Login, Dashboard, Feeding Log, Expense Log, Reports.

Applied consistent color scheme (green/yellow for farming theme) and modern typography.

Converted wireframes into Flutter layouts using Scaffold, AppBar, TextFormField, and ListView.

Implemented navigation between screens using Navigator.pushNamed.

Added form validation for Feeding and Expense entries.

Tested UI flow on emulator and Android device.

Screenshots



Figure 12: Figma wireframes showing all 5 screens side-by-side.



Figure 13: Login screen wireframe in Figma.

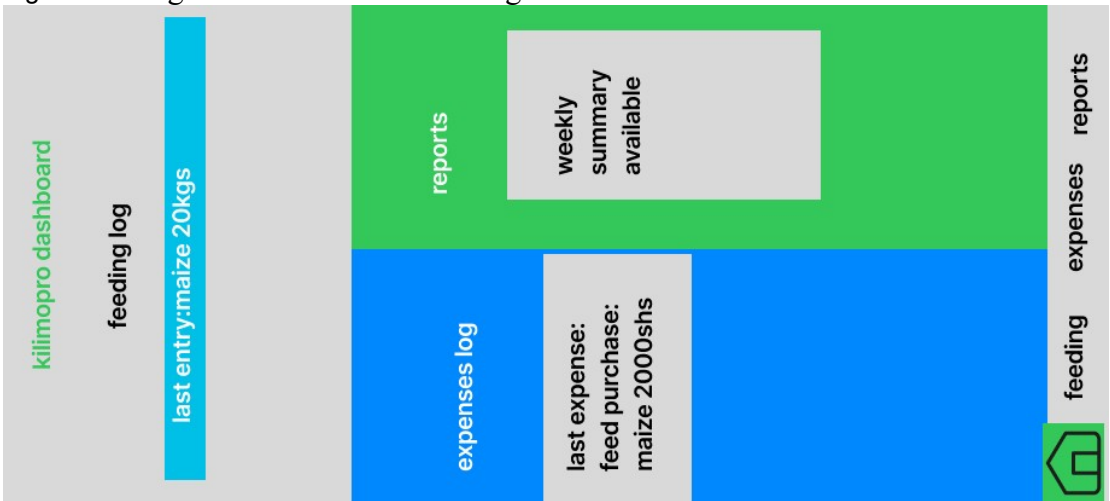


Figure 14: Dashboard wireframe in Figma.

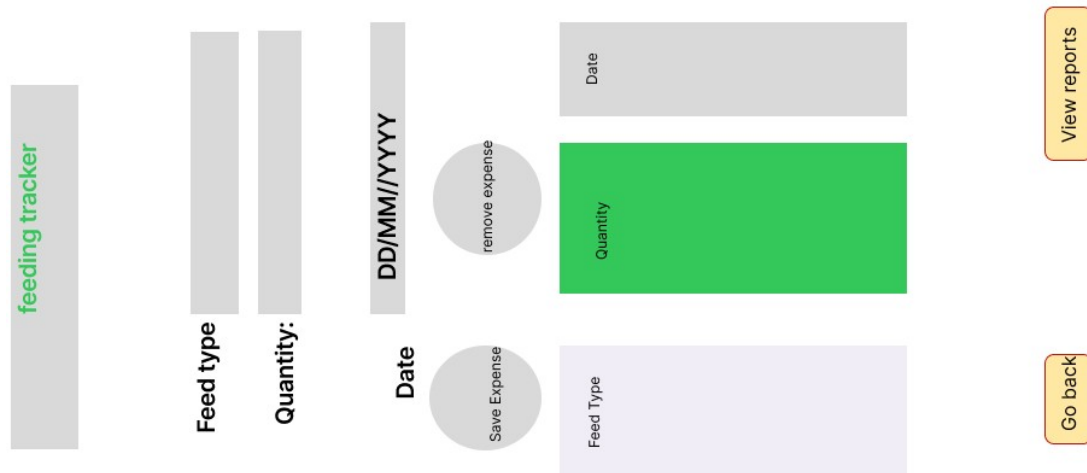


Figure 15: Figma Wireframe Feeding Log form.

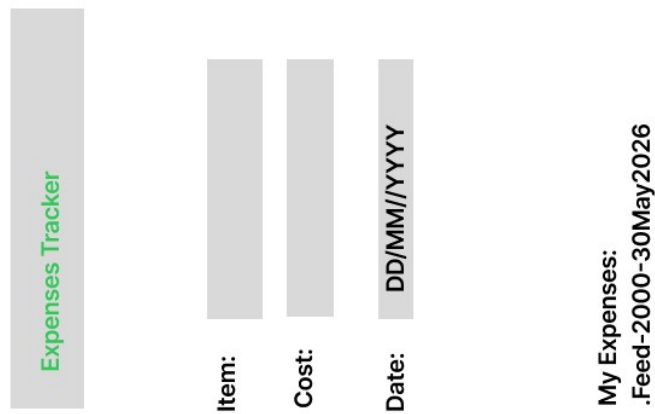


Figure 16: Figma Wireframe Expense Log form.

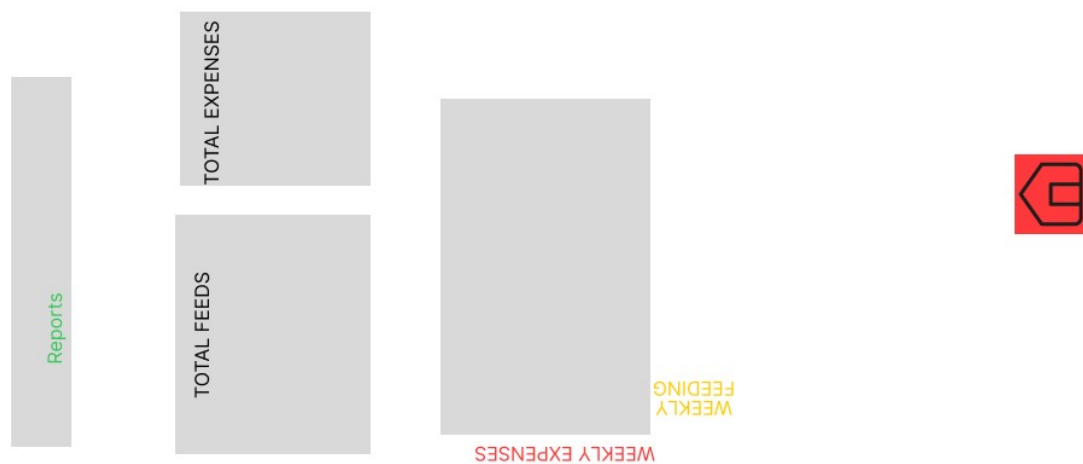


Figure 17: Figma Wireframe Reports screen with sample data.

Figure 8: Flutter code snippet for navigation and layout.

Challenges Faced

Limited Figma free plan (only 3 pages) → solved by using multiple frames on one page.

Ensuring responsive design across emulator and real device.

Handling form validation and navigation consistency.

Solutions Implemented

Organized all wireframes as frames within one Figma page.

Used MediaQuery in Flutter for responsive layouts.

Applied TextFormField validators for input validation.

Debugged navigation routes to ensure smooth flow.

Outcome

KilimoPro prototype completed with 5 functional screens.

Navigation works between login, dashboard, feeding log, expense log, and reports.

UI design is consistent, responsive, and aligned with farming theme.

Reflection

Week 3 marked the transition from practice apps to the real project (KilimoPro).

Learned how to bridge design (Figma) with implementation (Flutter).

Gained experience in UI/UX design, validation, and responsive layouts.

Ready to move into database integration and deployment in later weeks.